Jails: host-access policy

TBK SPEC 08:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 08:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. Policy model (docker -v semantics) 4

2. Semantics 5

2.1 The platform floor (locked 2026-08-14) 6

3. Enforcement point (the single choke) 7

Shipped semantics (crates/tfs, roadmap 09) 8

4. Manifest integration 9

Shipped surfaces (roadmap 35) 9

5. Scope (honest, locked 2026-07-26) 9

6. Executor-layer trust (locked 2026-07-27) 10

7. Conformance: the attack matrix (locked 2026-10-23) 11

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Normative specification of host-filesystem access control for running payloads. Status: SHIPPED (roadmaps 09 + 30 + 35) — the TFS enforcement point (§1–§3) is SHIPPED in `crates/tfs` (policy model, `tebako_fs_host_policy`, per-route EPERM/EROFS gating, symlink re-validation, jail acceptance suite); the `TEBAKO_JAIL` env form of the policy (§1) is SHIPPED with roadmap 30's preload shim (`tfs exec --jail` carries it to native binaries); manifest integration (§4 — `capabilities.host` in `crates/tpkg`), the press/dispatch surfaces (`tebako press --jail` writing the package manifest's `jail:` block, `tebako run/tebako-shim --jail|--mount|--no-host`), the bootstrap's manifest∩user composition at handoff, and the audit-journal logging of violations (`event=jail-deny`) are SHIPPED with roadmap 35. Enforcement exists because every file access already flows through the TFS layer — one choke point covers every consumer.

1. Policy model (docker -v semantics)

FIGURE 1

```
jail:
  default: open | deny          # open = today's behavior
(cwd + writes pass through)
  mounts:
    - host: $HOME/sources
      mount: /work
      access: ro | rw
  argument_files: auto-allowed  # the input file you hand
the command is allowed even under deny
```


© 2026 The Tebako Project

Three profiles: . **full access** (open) — host cwd + writes pass through (today). . **directory-scoped** — cwd mapped to `/work`, rw; nothing else. . **file-scoped tight jail** (deny + argument files only) — run an untrusted payload against one input, nothing else.

The `TEBAKO_JAIL` env form (the form the runtime driver and the preload shim consume — the loader's wire spelling of a policy). ABNF:

FIGURE 2

```
policy      = default *(";" grant)
default     = "open" / "deny"
grant       = host-grant / argfile-grant
host-grant  = host-path ":" mount-path ":" access
host-path   = 1*(%x21-3A / %x3C-7E)        ; any visible
char except ';'
mount-path  = 1*(%x21-3A / %x3C-7E)        ; the in-VFS
point, same alphabet
access      = "ro" / "rw"
argfile-grant = "@" 1*(%x21-7E)            ; an argv file
path, read-allowed
```

open grants host cwd + writes (today's behavior); deny is the default-deny floor. Each `;host:mount:access` grants the host path at the mount point with the named access; `;@path` auto-allows an argument file even under deny. The grammar's alphabet excludes `;` (the grant separator); a windows drive-qualified floor spelling is NOT carried (the right-split cannot — every consumer re-derives the floor at its own bind, §2's inheritance rule). Examples: deny`;/home/u/src:/work:rw`, deny`;/home/u/src:/work:rw;@/home/u/in.csv`, open.

2. Semantics

- Writes to an `ro` mount fail `EROFS`; paths outside every mount fail `EPERM`. Payload images themselves are always ro (spec 11).
- Symlinks resolve inside the granted view: realpath checked at bind time, re-validated on each open.
- **Precedence (locked):** the package's manifest REQUESTS access (`capabilities.host`); the user can always TIGHTEN it (`tebako run --jail pkg`, `--mount src:/work:rw`, `--no-host`) — user policy always wins. Signed/trusted payloads may get wider latitude by policy choice, never by bypass.
- Violations are logged to the tebako audit journal with path + syscall class.

© 2026 The Tebako Project

2.1 The platform floor (locked 2026-08-14)

Every policy bound under the deny default gains the **platform floor**: a fixed per-platform set of read-only grants covering the surface a spawned interpreter or runtime physically cannot boot without. The floor exists because the failure mode for a missing platform grant was never a policy verdict — under a scratch-only jail (`deny ; <scratch> : <scratch> : rw`) the JVM crashed with a SIGSEGV at `getMacOSXLocale` (its locale/framework init NULL-derefs when reads under `/usr` are denied; spec 22 phase-E dogfood, 2026-08-13). A jail whose denial crashes the child in someone else's library is wrongly constructed: denying the platform surface never produced a working process, so the floor grants it always.

- **Entries (evidence-driven):**
  - macOS — `/usr`, `/System`, `/Library` (the proven set above).
  - windows — `%SystemRoot%\System32` (the loader's DLL root; no process resolves its imports without it), `%SystemRoot%\SysWOW64` (the 32-bit view for 32-bit children), `%SystemRoot%\Fonts` (the GDI font tree GUI runtimes enumerate — the JVM's AWT init reads it). `%SystemRoot%` resolves at bind (`C : \Windows` when unset).
  - Other unix — none today; an entry joins a list only with a proven platform-process consumer, cited by run. The lists are amended by evidence, never by anticipation.
- **Supersede rule:** an authored mount covering a floor path (same or an ancestor prefix) supersedes the floor entry — the floor never narrows what the author allowed. Widening a floor path to `rw` is therefore one authored grant away; there is no way to drop a floor path short of not denying (see below) — that is the point of a floor.
- **Bind semantics:** the floor applies at `HostPolicy : : bind` (§3's single choke), only under the deny default — under open it grants nothing the default does not already allow, so the no-policy path stays byte-identical (`never_denies` keeps its exact meaning). A floor path absent on the host is skipped silently (it is a courtesy surface, not an authored request whose absence must fail the bind).
- **Ancestor traverse (locked 2026-08-14; extended 2026-10-08):** every bound grant — authored mount, floor entry, or `@file` argument-file grant — implies its strict ancestors are traversable: each ancestor answers an exact-path READ — never a prefix (no sideways exposure), never a write. Canonicalization walks are universal: the JVM reads its cwd and every ancestor at VM init, and the factory's jailed_exec leg died with "Could not determine current working directory" when the chain was denied (PR #95 macOS legs). The `deny : arg` profile's file grants join the same rule (2026-10-08, tebako#539): a granted input file you cannot stat your way to is unusable — the JVM died at VM init under `deny : arg` with a granted file in a denied cwd chain. The set is derived at bind — never authored, never serialized. It is what makes an authored `$CWD`-style grant sufficient in practice: the payload names the directory it works in; the platform walk to reach it passes by construction.
- **Inheritance:** floor mounts are NOT serialized into `TEBAKO_JAIL` — the env grammar's `host : mount : ro | rw` right-split cannot carry a windows drive-qualified floor spelling,

© 2026 The Tebako Project

and it never needs to: every consumer re-derives the floor at its own bind (supersede makes that idempotent), so a spawned child's inherited spec plus its bind yields a policy identical to the parent's. Enforcement treats authored and floor mounts as one longest-prefix set.

- **The boundary (what never joins):** the floor is SYSTEM surface only. The workload's own tool tree (a JRE, a third-party install) and the user's home stay authored grants — an operator's deny must not silently read-expose private data, and the prefix grammar cannot express the "stat-only" grant the macOS CFPreferences home probe needs (spec 22 §3.4's journal-pinned chain, 2026-08-14: floor → named `jvm.cfg` error → named `InternalError` → boot with `<jre>:ro`

  — `<home>:ro`). The floor's promise is the end of the segfault

  class: every missing grant surfaces as the workload's own named error, pinned in the audit journal — never a crash in someone else's library.

3. Enforcement point (the single choke)

New TFS C ABI entry:

FIGURE 3

```
int tebako_fs_host_policy(int default_open /* 0 = deny */,
                          const tebako_host_mount_t* mounts,
size_t n_mounts,
                          const char* const* arg_files, size_
t n_arg_files);
```

- The host-passthrough path of every IO route consults the policy: prefix match + access bit per mount. `tebako_fs_stat/open/opendir/…` return EPERM (denied) / EROFS (ro write).
- The policy is about HOST paths; memfs mounts are unaffected.
- Implemented in the Rust TFS (`crates/tfs`) — the same layer that owns routing, so bootstrap, runtime driver, `tebako run`, and `tfs` all enforce identically with no per-app work.

© 2026 The Tebako Project

Shipped semantics (crates/tfs, roadmap 09)

- The policy is process state, not namespace state: `tebako_fs_unmount` does not reset it (fail-closed); a later `tebako_fs_host_policy` call replaces it. Install it after the payload mounts are established — the mount family's image read is itself policy-gated once a policy is active.
- Gated host-passthrough decisions: open (read and write modes), `stat`, `opendir`, `dlmap2file`, `extract_all`'s destination (a host write), and the mount family's image read. An allowed host path keeps the historic answer ENOENT ("not ours, pass through") for reads AND writes — under open a host write attempt therefore answers ENOENT, not EROFS, so the consumer's pass-through can perform the write (profile 1: "cwd + writes pass through"). Held memfs content is byte-for-byte unchanged (write opens on it stay EROFS).
- The per-process materialization tree is NOT a host path for policy purposes (tebako#502, 2026-08-30): a dlmap-marker path (…`/tebako-dl-<hex>/`… — closure copies and the home-layout whole-tree roots `tebako-home-<handle>/`…) the image does not answer is the runtime's own extraction served by its host copy. Every route (open/`stat`/`opendir`/`dlmap2file`) answers it ENOENT without consulting the policy: the hex leaf is per-process (unauthorable at bind — the path does not exist to canonicalize), the content rode in through the ungated extraction writes, and the tree is removed at process exit. This is what lets a wrapper-pattern runtime (spec 22 §3's exec cache) compose with a deny jail.
- "Host path" means any path the mounts do not hold — outside every mount, or covered by a mount but absent from its image (spec 11 §2): the app payload mounts at `/` and the host filesystem stays reachable exactly through this rule.
- The consumer must route, not just the engine (tebako#917): a runtime whose IO shims call this family only for the paths the mounts COVER hands every uncovered path to the kernel, where this policy cannot see it. Such a shim asks `int tebako_fs_host_guard(void)` — 1 when the installed policy judges an uncovered path (a deny or record default) — and routes every absolute path through the family while it answers 1. An allowed path still answers ENOENT, the pass-through signal these shims already honour, so an unguarded run is unchanged and a guarded one denies where it said it would. Coverage is a namespace test; a namespace test is not an enforcement point.
- A mount's access bit applies even under an open default (an ro bind is ro in an otherwise open namespace, docker-style); longest host prefix wins on path-component boundaries; argument files are an exact read-only grant, even under deny.
- Bind-time realpath covers mount sources and argument files (a missing one fails the install with its errno); every gated decision re-canonicalizes the target, so symlinks swapped in after install resolve to their target and escapes fail EPERM.

© 2026 The Tebako Project

4. Manifest integration

The payload manifest (spec 03) gains `capabilities.host` — the access the payload was built to need (e.g. metanorma: read the input file's directory, write the output directory). Dispatch surfaces (spec 07) compose: manifest request ∩ user policy = effective jail.

Shipped surfaces (roadmap 35)

- Model: `crates/tpkg/src/jail.rs` owns the authored shape (YAML block ↔ `TEBAKO_JAIL` env form ↔ `--jail` cli spec: `open | deny | deny:arg | a YAML file | the env grammar`) and the locked precedence algebra (`intersect` / `effective`): deny defaults win; each side's grants are capped by the other side's allowance at the same prefix (ro is sticky, a user `--no-host` drops request grants); argument files union and `auto-allowed` resolves argv files into read-only grants at dispatch. The payload manifest's `capabilities.host` and the package manifest's `jail:` block are both this type.
- Press: `tebako press --jail <spec>` writes the policy into the type-2 package manifest's `jail:` block (spec 02 §5b / spec 03 §6); block-less packages are byte-identical to before.
- Bootstrap: composes package `jail:` ∩ `TEBAKO_JAIL` at handoff and exports the effective policy as `TEBAKO_JAIL` (+ `TEBAKO_JAIL_SOURCE` = `manifest | user | manifest+user`, + `TEBAKO_JAIL_JOURNAL` → `$TEBAKO_HOME/journal.log`). Malformed policy fails closed, exit 73 (`EX_TEBAKO_JAIL`). No policy: nothing exported (legacy byte-identical).
- Dispatch: `tebako run <pkg> [--jail <spec>] [--mount <host:mount:ro|rw>]... [--no-host] [--] [args...]` and the same flags on `tebako-shim` (consumed before `--`; the payload mirror carries `capabilities.host` from install time).
- Audit journal (§2): every denial in the TFS layer appends `<ts> event=jail-deny path=<p> op=read|write source=<s>` to the tebako journal (`TEBAKO_JAIL_JOURNAL` > `$TEBAKO_HOME/journal.log` > `~/.tebako/journal.log`), best-effort — journaling never fails the operation it audits.

5. Scope (honest, locked 2026-07-26)

Jails are FILESYSTEM-ONLY. `capabilities.network` is advisory metadata (displayed by the info surface, spec 15), not an enforcement promise. Coverage: every IO route through TFS — all interpreted payloads AND native binaries running under the preload interposition shim (spec 07 §8 tier 1, whose IO flows through the same `host_policy`). NOT covered: statically-linked binaries (direct syscalls, no interposition point) and any

© 2026 The Tebako Project

process that escapes the shim — stated honestly in the manifest and docs. OS-level network confinement (seatbelt/seccomp/WFP) is a possible later layer, never claimed today.

The shim's coverage is SYMBOL-level (tebako#439): a jail holds only for the libc entry points the shim interposes — the named list in `crates/libtfs-preload` (lib.rs "Interposed surface"), which includes the LFS64/fortify/versioned twins (`open64`/`openat64`/`fopen64`, `__read_chk`/`__openat_2`, the `__xstat` family through `__fxstatat64`) because glibc hands `_FILE_OFFSET_BITS=64`/`_FORTIFY_SOURCE`/pre-2.33 callers DISTINCT symbols that never touch the plain names (the #439 hole: OpenSSL's `o_fopen.c` defines `_FILE_OFFSET_BITS=64` itself, so every `BIO_new_file`/`X509_LOOKUP_load_file` bound `fopen64` straight to libc). The WRITE class rides the same gate (tebako#444, closing the pre-existing hole the #439 read-class audit left): `open`/`openat` with `O_WRONLY`/`O_CREAT`/`O_RDWR` were policy-gated by the engine's open from the start, and `fopen`'s write/append/update modes (`w`/`a` bases, every `+` update form including `r+`, the `x`-forms) plus `creat` now route through `host_policy` BEFORE the real libc call touches the host — a jailed write outside an rw grant is EPERM by name, a write against an ro grant EROFS, and a write naming a memfs-held path is EROFS under any policy (payload images are always read-only). Known-uncovered at this layer: the remaining fortify open family (`__open_2`/`__open64_2`/`__openat64_2` — no importer observed in the 0.16.6 linux-gnu runtime) and raw `syscall(2)` IO. Separately, the runtime EXE's own statically-linked native libraries are gated only for symbols the exe itself defines (spec 22 §2: `dlopen`/`dlerror` today — defining the file-IO family in the exe is follow-up wiring in the runtime factory, tebako-runtime-ruby); an interpreter's patched IO and preload-delivered children are covered.

6. Executor-layer trust (locked 2026-07-27)

Tebako is the EXECUTOR of everything it runs — the parent that spawns and owns the child. Mediating a child's syscalls (preload interposition for dynamic children, seccomp user-notification for static children on Linux) is the execution contract, not hijacking: a hostile party would have to become the executor or the kernel to hold the same position, and both are outside the jail's threat model by definition. Jails bind the CHILD downward; they claim nothing about the executor or kernel upward.

© 2026 The Tebako Project

7. Conformance: the attack matrix (locked 2026-10-23)

A jail is only as good as the attacks it has been shown to stop. Every vector below is either **blocked and proven by a named test**, or **named out of scope with the reason** — there is no third state. The host-filesystem vectors live in `crates/tfs/tests/jails.rs`; the environment vector lives beside its implementation (`crates/tebako-driver/src/env_scrub.rs`). Both run on every platform leg of every CI run and therefore on every release: a regression reddens the release matrix, not a dashboard nobody reads.

TABLE 1

#	vector	verdict	the proof
1	lexical escape: `..`, doubled separators, `.` components inside a grant	blocked	`dot_dot_paths_normalize_lexically`
2	symlink escape: a link inside a granted tree pointing at a denied sibling (directory and file level)	blocked (EPERM)	`symlink_escape_attempt_fails`
3	TOCTOU: the tree is swapped for a symlink AFTER the policy was bound	blocked (EPERM)	`symlink_swap_after_bind_fails_on_revalidation` — every open re-validates through realpath
4	case folding: a denied path asked for with different case on a case-insensitive host	blocked (EPERM)	`case_variants_of_a_denied_path_are_denied` — the decision is taken on the canonical path (mac OS `realpath` returns the on-disk case), so it never depends on what the filesystem thinks about case
5	write into a held (image) tree	blocked (EROFS)	`ro_mount_refuses_writes_with_erofs`
6	write outside every grant under an rw mount	blocked (EPERM)	`directory_scoped_mount_rw_works_and_sibling_denies`
7	enumeration of a denied host directory	blocked (EPERM, nothing listed)	`deny_all_cannot_enumerate_or_read_but_memfs_is_unaffected`
8	reads/stats of a denied host file	blocked (EPERM, never ENOENT — the denial is a decision, not a miss)	`deny_all_cannot_enumerate_or_read_but_memfs_is_unaffected`
9	extract-all as an ungated host write	blocked (gated)	`extract_all_is_a_gated_host_write`
10	covered-but-not-held path under a `/` mount (the mount must not become a hole)	gated (the host decision runs)	`covered_but_not_held_paths_fall_through_to_the_host_decision`

© 2026 The Tebako Project

#	vector	verdict	the proof
11	image reads through the policy	gated	`mount_family_reads_the_image_through_the_policy`
12	unmount under a live policy	fail-closed	`policy_survives_unmount_fail_closed`
13	malformed policy spec	fail-closed (named)	`policy_install_validates_its_arguments`
14	argument-file grant: only the declared file, nothing beside it	blocked	`tight_jail_allows_only_the_argument_file`
15	packager/CI credentials reaching the payload: `TEBAKO_GITHUB_TOKEN` and every other packager-side `TEBAKO_*` variable past the handoff	blocked (blanked, never logged; `env_pass` is the explicit opt-in)	`the_packager_token_is_scrubbed_and_the_contract_kept`, `the_env_pass_opt_in_carries_its_declared_names`, `the_factory_smoke_contract_survives_the_scrub` (spec 03 §2.10, tebako#737)
16	Windows alternate data stream (`file.txt:stream`) on a denied base	PLANNED — the decision rides the base path (a stream suffix is not a path component), and the proof lands with the windows leg that can exercise it	— (named, not claimed)
17	statically-linked binary, or raw `syscall(2)` IO	OUT OF SCOPE — no interposition point; §5 states it in the manifest and the docs	§5
18	fortify open family still uncovered (`__open_2`/`__open64_2`/`__openat64_2`)	OUT OF SCOPE — no importer observed in the shipped runtimes; the symbol list is the interposed surface's SSOT	§5
19	the runtime EXE's own statically-linked stat/open definitions	OUT OF SCOPE today — gated only for the symbols the exe defines (spec 22 §2); follow-up wiring lives in the runtime factory	§5
20	a payload's own sockets (network confinement)	OUT OF SCOPE today — `capabilities.network` is advisory metadata; the network axis is SOTA arc 05's second item, not a claim of this spec	§5

Row 16 is the one open vector that is IN scope and not yet proven: it is named here so nobody reads the matrix as "done", and it does not close until a windows test stands behind it.

© 2026 The Tebako Project